॥वसुधैव कुटुम्बकम्॥

Symbiosis Institute of Technology, Pune

Faculty of Engineering

CSE - Academic Year 2026-27

Java Programming – Lab (Batch 2025-29)

Lab Assignment No:- 28

Name of Student	Abobakr Mohamed Elamen
PRN No.	25070122015
Batch	2025-29
Class	Programming with JAVA
Academic Year & Semester	2nd Year – Sem III
Title of Assignment	Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.
Practice questions	1. Create a remote calculator service using Java RMI.

CalculatorService.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface CalculatorService extends Remote {
    double add(double a, double b) throws RemoteException;
    double subtract(double a, double b) throws RemoteException;
    double multiply(double a, double b) throws RemoteException;
    double divide(double a, double b) throws RemoteException;
}
```

CalculatorServiceImpl.java

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class CalculatorServiceImpl extends UnicastRemoteObject
implements CalculatorService {

    public CalculatorServiceImpl() throws RemoteException {
```

```
        super();
    }

    public double add(double a, double b) throws RemoteException {
        return a + b;
    }

    public double subtract(double a, double b) throws
RemoteException {
        return a - b;
    }

    public double multiply(double a, double b) throws
RemoteException {
        return a * b;
    }

    public double divide(double a, double b) throws
RemoteException {
        if (b == 0) {
            throw new ArithmeticException("Cannot divide by
zero");
        }
        return a / b;
    }
}
```

CalculatorServer.java

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorServer {
    public static void main(String[] args) {
        try {
            CalculatorService service = new
CalculatorServiceImpl();
            Registry registry =
LocateRegistry.createRegistry(1099);
            registry.rebind("CalculatorService", service);
            System.out.println("Calculator server is running...");
        } catch (Exception e) {
            System.out.println("Server error: " + e.getMessage());
        }
    }
}
```

CalculatorClient.java

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorClient {
    public static void main(String[] args) {
        try {
            Registry registry =
LocateRegistry.getRegistry("localhost", 1099);
            CalculatorService calc = (CalculatorService)
registry.lookup("CalculatorService");

            double a = 20, b = 5;
            System.out.println("Numbers: " + a + " and " + b);
            System.out.println("Addition       : " + calc.add(a,
b));
            System.out.println("Subtraction    : " +
calc.subtract(a, b));
            System.out.println("Multiplication : " +
calc.multiply(a, b));
            System.out.println("Division       : " +
calc.divide(a, b));
        } catch (Exception e) {
            System.out.println("Client error: " + e.getMessage());
        }
    }
}
```

2. Create a Java application using RMI where the server converts temperature values between Celsius and Fahrenheit. The client sends input temperature and receives converted outputs.

TermperatureService.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface TemperatureService extends Remote {
    double celsiusToFahrenheit(double celsius) throws
RemoteException;
    double fahrenheitToCelsius(double fahrenheit) throws
RemoteException;
}
```

TermperatureServiceImpl.java

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class TemperatureServiceImpl extends UnicastRemoteObject
implements TemperatureService {

    public TemperatureServiceImpl() throws RemoteException {
        super();
    }

    public double celsiusToFahrenheit(double celsius) throws
RemoteException {
        return (celsius * 9 / 5) + 32;
    }

    public double fahrenheitToCelsius(double fahrenheit) throws
RemoteException {
        return (fahrenheit - 32) * 5 / 9;
    }
}
```

TermperatureServer.java

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class TemperatureServer {
    public static void main(String[] args) {
        try {
            TemperatureService service = new
TemperatureServiceImpl();
            Registry registry =
LocateRegistry.createRegistry(1099);
            registry.rebind("TemperatureService", service);
            System.out.println("Temperature server is
running...");
        } catch (Exception e) {
            System.out.println("Server error: " + e.getMessage());
        }
    }
}
```

TermperatureClient.java

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;
```

```java
public class TemperatureClient {
    public static void main(String[] args) {
        try {
            Registry registry =
LocateRegistry.getRegistry("localhost", 1099);
            TemperatureService service = (TemperatureService)
registry.lookup("TemperatureService");

            Scanner sc = new Scanner(System.in);
            System.out.println("1. Celsius to Fahrenheit");
            System.out.println("2. Fahrenheit to Celsius");
            System.out.print("Enter choice: ");
            int choice = sc.nextInt();
            System.out.print("Enter temperature: ");
            double temp = sc.nextDouble();

            if (choice == 1) {
                System.out.println(temp + " C = " +
service.celsiusToFahrenheit(temp) + " F");
            } else if (choice == 2) {
                System.out.println(temp + " F = " +
service.fahrenheitToCelsius(temp) + " C");
            } else {
                System.out.println("Invalid choice");
            }
        } catch (Exception e) {
            System.out.println("Client error: " + e.getMessage());
        }
    }
}
```

Output

1)

```
C:\Users\bkor\OneDrive\Desktop\JAVA_PRACTICALS\JAVA_EXP_28>javac *.java

C:\Users\bkor\OneDrive\Desktop\JAVA_PRACTICALS\JAVA_EXP_28>java CalculatorServer
Calculator server is running...
```

```
C:\Users\bkor\OneDrive\Desktop\JAVA_PRACTICALS\JAVA_EXP_28>java CalculatorClient
Numbers: 20.0 and 5.0
Addition       : 25.0
Subtraction    : 15.0
Multiplication : 100.0
Division       : 4.0
```

2)

```
C:\Users\bkor\OneDrive\Desktop\JAVA_PRACTICALS\JAVA_EXP_28>javac *.java

C:\Users\bkor\OneDrive\Desktop\JAVA_PRACTICALS\JAVA_EXP_28>java TemperatureServer
Temperature server is running...
```

	
Conclusion	RMI lets a Java program call methods on an object running in another JVM as if it were local. The remote interface defines what the client can call, the server registers the implementation in the RMI registry, and the client looks it up by name. The stub on the client side handles marshalling the call and the result over the network. Examples like the calculator and the temperature converter show how distributed applications can share services without the programmer writing any low-level socket code.